

LOCAL SYSTEMS

Build Your Own AI Coding Agent

LLM Chat vs. Agent Loops

Standard LLM Chat

- One Request → Response loop
- Confined strictly to text chat bubble
- No access to system terminal or tools
- Cannot run code or verify test cases
- Requires constant copy-paste of files

Autonomous Coder Agent

- Iterative Parse-Plan-Act-Reflect cycle
- Runs system terminal shell commands
- Integrates deep local tool calling (MCP)
- Modifies files, compiles, and runs tests
- Self-corrects errors in the loop

VRAM & Speeds Sizing Matrix

- Sizing is critical to prevent unified system memory swapping.

VRAM / RAM	Autocomplete	Chat / Agent	Speed / Throughput
8 GB	Qwen 0.5B Base	Qwen 1.5B Inst	~188 tokens/sec
16 GB	Qwen 1.5B Base	Qwen 7B Instruct	~68 tokens/sec
24 GB	Qwen 1.5B Base	Qwen 14B Instruct	~30 tokens/sec
32 GB+	Qwen 3B Base	Codestral 22B	~28 tokens/sec

 **Swap Warning:** Overflowing physical memory pages to SSD swap collapses generation speed down to ~1.8 tokens/sec.

Architecture Flow

- **1. User Interface:** VS Code client + Continue Extension UI for active workspace integration.
 - **2. Agentic Framework:** Prompt sequencing context building, and tool orchestration loops.
 - **3. Model Context Protocol:** Decoupled JSON-RPC tool gateway communicating with local servers.
 - **4. Inference Server:** Ollama running locally at localhost:11434 serving model parameters.
 - **5. Compute Hardware:** Apple Silicon Unified Memory Bus driving GPU core operations.

IDE Integration Config

```
{
  "models": [{
    "title": "Qwen 14B Chat",
    "provider": "ollama",
    "model": "qwen2.5-coder:14b"
  }],
  "tabAutocompleteModel": {
    "title": "Qwen 1.5B Auto",
    "provider": "ollama",
    "model": "qwen2.5-coder:1.5b"
  }
}
```

Configure autocomplete and reasoning models on concurrent local threads.

SLIDE 6

Custom Agent Skills (Python)

```
# parse.py - Custom symbol parser skill
import sys, ast
def parse_exports(path):
    with open(path) as f:
        tree = ast.parse(f.read())
        funcs = [n.name for n in ast.walk(tree) if isinstance(n,
ast.FunctionDef)]
        print(f"Verified exports: {funcs}")
if __name__ == "__main__":
    parse_exports(sys.argv[1])
```

Expose custom system scripting skills to local agents via Model Context Protocol.

Workspace Sandboxing

- Autonomous file writes pose destructive code risks. Always enforce Git sandboxing rules:
 - **1. Clean Working Tree:** Verify `git status` is clear with zero uncommitted edits before starting.
 - **2. Directory Boundaries:** Restrict tool write permissions to the active project workspace.
 - **3. Interactive Audit:** Review agent edits block-by-block using `git diff --stat`.
 - **4. Rollback Plan:** Undo incorrect modifications instantly with `git reset --hard HEAD`.

SLIDE 8

Private AI Coder Sandbox

Reclaim source code privacy, eliminate telemetry leaks, and run private coder tools at zero token cost.

Get Code & Configs:

- **Substack:** praveenbuilds.substack.com
- **GitHub (Companion Code):**
github.com/praveenveera/software-permanence
- **Dev.to:** dev.to/praveen_builds
- **Medium:** medium.com/@praveenveera92
- **LinkedIn:** linkedin.com/in/praveen-veera-6ab22567